

LOVELY PROFESSIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

INT315 – CLUSTER COMPUTING
Project Report

Campus Placement Prediction and Student Employability Analysis Using Apache Spark MLlib and PySpark

Student Name: Putluru Om Sai Nandan Reddy
Registration Number: 12304471
Section: K3M2328
Roll No: 03
Course Code: INT315
Course Name: Cluster Computing
Faculty: PradeepKumar

Table of Contents

1. Problem Statement **2**

2. Objectives of the Project **3**

3. Dataset Description **3**

4. Dataset Source **4**

5. Tools and Technologies Used **4**

6. Implementation / Methodology **4**

6.1 Environment Setup 4

6.2 Dataset Loading 5

6.3 Data Preprocessing 5

6.4 Exploratory Data Analysis 5

6.5 Placement Classification 5

6.6 Salary Prediction 6

6.7 Student Clustering 6

7. Complete Code **6**

8. Execution Screenshots **21**

9. Output / Result Screenshots **23**

10. Result Analysis **25**

10.1 Classification Analysis 25

10.2 Salary Regression Analysis 26

10.3 Clustering Analysis 26

10.4 Overall Analysis 26

11. Conclusion **26**

1. Problem Statement

Placement is one of the main outcomes by which a technical programme is judged, both by students and by the institution. Studying which students get placed, and what salary they receive, can help a placement cell see where preparation is needed, for example in technical skills, communication, internships or academic backlogs. Placement outcomes are rarely decided by one factor. Academic percentages (SSC, HSC and degree), technical and soft-skill scores, aptitude, internships, projects, certifications, work experience and backlogs can all be associated with whether a student is placed, so the relationship is better studied with data than by guesswork.

The dataset used here has about 100,000 student records with 26 columns. Reading it, cleaning it, encoding the categorical columns and training several models on it involve repeated passes over the data. Apache Spark handles this kind of work through DataFrames and ML pipelines that split the data into partitions and process them in parallel. This is why Spark and PySpark were chosen for this project, even though the project itself ran in local mode on one machine.

Three analysis problems are addressed. First, placement outcome is predicted as a binary classification task so that students who are unlikely to be placed can be identified. Second, for students who were placed, the salary in LPA is estimated with regression models. Third, K-means clustering is applied to academic and skill attributes to look for groups of students with similar profiles. This project works only on one dataset and does not claim to solve the wider problem of campus recruitment; its findings describe this dataset only.

2. Objectives of the Project

1. **To implement distributed machine learning using Apache Spark and PySpark:** Process and analyze the 100,000 student records using Spark DataFrames and Spark ML pipelines, demonstrating scalable data processing in a cluster computing environment.
2. **To develop a placement classification model:** Train and compare Logistic Regression, Decision Tree, Support Vector Machine (SVM), and Naive Bayes models to predict whether a student is likely to be placed based on academic performance, skills, internships, projects, certifications, and other relevant attributes.
3. **To develop a salary prediction model:** Apply Linear Regression and Decision Tree Regression to estimate the expected salary in LPA for eligible students, and compare their predictive performance using RZ, RMSE, and MAE.
4. **To evaluate and optimize machine learning model performance:** Assess classification models using a confusion matrix, accuracy, precision, recall, and F1-score, and evaluate regression models using RZ, RMSE, and MAE to identify the most suitable models for placement prediction and salary estimation.
5. **To identify important placement factors and student profiles:** Perform exploratory data analysis, generate a correlation heatmap, analyze the influence of academic and skill-related attributes, and use K-means clustering to group students into meaningful employability profiles for targeted placement preparation.

3. Dataset Description

The dataset is called **Campus Placement Prediction**. The notebook reads it from a file named campus_placement_data.csv. After loading, df.count() returned 100,000 records and the schema printed by Spark shows 26 columns (Figure 2). Each row describes one student. The dataset is used in this project to train and compare classification models for placement, regression models for salary, and to run exploratory analysis and K-means clustering. The column names and data types in Table 1 are taken from the schema printed in the notebook output.

Group	Columns (as named in the dataset)	Spark type
Identifier (1)	student_id	string
Categorical (7)	gender, city_tier, ssc_board, hsc_board, hsc_stream, degree_field, specialization	string
Numerical – percentages and scores (8)	ssc_percentage, hsc_percentage, degree_percentage, mba_percentage, technical_skills_score, soft_skills_score, aptitude_score, communication_score	double
Numerical – counts (8)	age, internships_count, projects_count, certifications_count, work_experience_months, leadership_roles, extracurricular_activities, backlogs	integer
Classification target (1)	placed (0 = not placed, 1 = placed)	integer
Regression target (1)	salary_lpa (salary in lakhs per annum)	double

The 16 numerical columns and the 7 categorical columns are the input features, giving 23 features in all. student_id is only an identifier and is not used as a feature, and neither placed nor salary_lpa is included in the feature vector. The column placed is the label for classification. The column salary_lpa is the label for regression and is used only for students who were placed and have a salary above zero. In the five rows shown in Figure 3, the not-placed student has a salary of 0.0, and mba_percentage is 0.0 and specialization is "None" for all five. These five rows are too few to say how these two columns look in

the full dataset.

4. Dataset Source

Item	Details
Original dataset source	Kaggle dataset page titled “Campus Placement Prediction”, published under the Kaggle
Dataset download link	https://www.kaggle.com/datasets/likithagedipudi/campus-placement-prediction?resource=download
Dataset used for the project	The file campus_placement_data.csv, from a locally downloaded archive, read in the notebook from the path shown in Listing 2 (100,000 records, 26 columns).

The Kaggle page lists “synthetic” among its keywords, so the data is treated here as synthetic (generated) and not as records collected from real students. Only the page title, short description and keywords could be read when the page was checked, so the author’s full name, publication date and the method used to generate the data are not stated in this report.

5. Tools and Technologies Used

Technology	Role in this project (as seen in the notebook)
Apache Spark 3.5.9	Cluster computing engine; the session is created with master local[*] and 8 shuffle partitions.
PySpark/Spark DataFrames	Python API used to read the CSV, clean the data, filter, split, group and aggregate.
Spark MLlib (Spark ML)	Pipeline, Imputer, StringIndexer, OneHotEncoder, VectorAssembler, StandardScaler; LogisticRegression, DecisionTreeClassifier, LinearSVC, NaiveBayes; LinearRegression, DecisionTreeRegressor; KMeans; RegressionEvaluator, ClusteringEvaluator.
Python 3.10.0, Jupyter Notebook	Implementation language and the interactive environment (Python 3 ipykernel).
Pandas, NumPy	Pandas holds the metric tables and the small samples converted from Spark for plotting; NumPy is imported.
Matplotlib, Seaborn	Donut chart, box plot, correlation heatmap, performance comparison plot, confusion matrix and actual-versus-predicted scatter plot.
Scikit-learn	Used only for the classification metrics (accuracy, precision, recall, F1-score) and the confusion matrix, computed on the predictions collected from Spark.

6. Implementation / Methodology

The steps below follow the order of the notebook cells. The full code is given in Section 8 and the outputs in Sections 9 and 10.

6.1 Environment Setup

The first cell imports the Python libraries (NumPy, Pandas, Matplotlib, Seaborn) and the required PySpark modules for DataFrame functions, ML pipelines, feature transformers, classifiers, regressors, K-means and evaluators. Seaborn’s "whitegrid" theme is set for plots. A SparkSession named "CampusPlacementPrediction" is created in local mode with 8 shuffle partitions.

6.2 Dataset Loading

The CSV file is read with spark.read.csv using header=True and inferSchema=True, so Spark takes column names from the first row and guesses the data types. The notebook prints the record count (100,000) and the schema with 26 columns.

6.3 Data Preprocessing

- **Duplicates:** dropDuplicates() is applied. The record count printed after cleaning is 100,000, the same as before, so no duplicate rows were removed.
- **Feature lists:** 16 numerical and 7 categorical columns are defined as input features.
- **Type conversion and invalid values:** the numerical columns and salary_lpa are cast to double, and NaN or infinite values are replaced by null. placedis cast to double and only rows with a label of 0.0 or 1.0 are kept. The count is at 100,000, so no row was dropped here.
- **Missing values:** the notebook does not print a separate null count. Missing categorical values are filled with "Unknown", and missing numerical values are handled by a median Imputer inside the pipeline, which learns the medians from the training data only.
- **Train-test split:** randomSplit([0.8, 0.2], seed=42) gives 80,200 training and 19,800 testing records (Figure 3).
- **Encoding and feature vector:** the function make_preprocessing_stages() returns the stages Imputer, seven StringIndexers (handleInvalid="keep"), OneHotEncoder and VectorAssembler, which builds a single features vector from the 16 imputed numerical columns and 7 encoded categorical columns. These stages are placed at the start of every classification and regression pipeline. The numerical features are not scaled in these pipelines.

6.4 Exploratory Data Analysis

Three charts were made. (i) A donut chart of the placement outcome (Figure 7) shows 84.4% placed and 15.6% not placed, so the classes are imbalanced. (ii) Box plots of SSC, HSC and degree percentage for placed and not-placed students, drawn from a 25% random sample (Figure 8), show a higher median for placed students at all three levels, although the boxes overlap a lot. (iii) A Pearson correlation heatmap of 13 columns, also from a 25% sample (Figure 9), shows that placed has its strongest correlations with backlogs (−0.26), soft_skills_score (0.24), communication_score (0.24) and technical_skills_score (0.23). All correlations with the target are weak, with an absolute value of at most 0.26. SSC, HSC and degree percentages are highly correlated with one another (0.67 to 0.83), as are the three skill scores (0.73 to 0.92). Correlation between the academic group and the skill group is close to zero.

6.5 Placement Classification

Each classifier is wrapped in a Spark Pipeline after the preprocessing stages, fitted on train_df, and used to predict on test_df. The placed and prediction columns of the test set are collected into Pandas and the metrics are calculated with scikit-learn, using class 1 (placed) as the positive class. The model with the highest F1-score is taken as the best model and its confusion matrix is plotted.

Model	Spark class	Settings used
Logistic Regression	LogisticRegression	maxIter = 50
Decision Tree	DecisionTreeClassifier	maxDepth = 5, seed = 42
SVM	LinearSVC (linear SVM)	maxIter = 50
Naive Bayes	NaiveBayes	modelType = "multinomial"
Linear Regression (salary)	LinearRegression	maxIter = 50
Decision Tree Regression (salary)	DecisionTreeRegressor	maxDepth = 5, seed = 42

Metrics. The confusion matrix counts correct and wrong predictions for each class: true negatives (TN), false positives (FP), false negatives (FN) and true positives (TP). Accuracy = (TP+TN)/all predictions. Precision = TP/(TP+FP), the share of students predicted as placed who were actually placed. Recall = TP/(TP+FN), the share of actually placed students that the model found. F1-score is the harmonic mean of precision and recall.

6.6 Salary Prediction

The target is salary_lpa. Only students with placed = 1 and a non-null salary greater than 0 are used, which gives 84,432 students. This subset is split 80:20 with seed=42. Linear Regression and Decision Tree Regression are each placed after the same preprocessing stages, fitted on the salary training set and evaluated on the salary test set with Spark's RegressionEvaluator. The model with the lowest RMSE is selected, and a scatter plot of actual versus predicted salary (up to 2,000 sampled test points, with a dashed line for perfect prediction) is drawn for it.

Metrics. RMSE is the square root of the mean squared error and is in LPA units; large errors are punished more. MAE is the mean absolute difference between actual and predicted salary, also in LPA. R^2 is the share of the variance in salary explained by the model; a value of 1 is a perfect fit, and a value near 0 means the model is no better than predicting the mean.

6.7 Student Clustering

K-means uses 12 columns: SSC, HSC and degree percentages, technical, soft, aptitude and communication scores, internships, projects, certifications, work experience in months, and backlogs. They are combined by a VectorAssembler and standardized with StandardScaler (mean removed and unit standard deviation) so that columns with large values do not dominate the distance. The scaler is fitted on all rows of the cleaned data that have no null in these columns. K-means is run for K = 2, 3, 4 and 5 with seed=42, the silhouette score is calculated for each K, and the K with the highest silhouette score is chosen (Table 7). The purpose of clustering is to group students who look alike on academic and skill measures, so that preparation can be planned for each group. The notebook stops after choosing K: it does not fit a final model with the chosen K or describe the clusters.

7. Complete Code

The listings below are the code cells of Campus_Placement_Prediction-Copy1.ipynb, copied without any change and in the original order. Listing numbers match the notebook cell numbers counted from 1; the last cell of the notebook is empty and is not shown. Line numbers restart in each listing.

Listing 1 (notebook cell 1): Imports and Spark session (local mode)

```
import os
import zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import DoubleType, StringType
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    StringIndexer,
    OneHotEncoder,
    VectorAssembler,
    StandardScaler
)
from pyspark.ml.classification import (
    LogisticRegression,
    DecisionTreeClassifier,
    LinearSVC,
```

```
    NaiveBayes
)
from pyspark.ml.regression import (
    LinearRegression,
    DecisionTreeRegressor
)
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import (
    MulticlassClassificationEvaluator,
    RegressionEvaluator
)
sns.set_theme(style="whitegrid")
spark = (
    SparkSession.builder
    .appName("CampusPlacementPrediction")
    .master("local[*]")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR")
print("Spark Version:", spark.version)
```

Listing 2 (notebook cell 2): Loading the CSV file into a Spark DataFrame

```
csv_path = r"C:\Users\putlu\Downloads\archive (2)\campus_placement_data.csv"
df = spark.read.csv(
    csv_path,
    header=True,
    inferSchema=True
)
print("Total records:", df.count())
df.printSchema()
```

Listing 3 (notebook cell 3): Cleaning, feature lists and type conversion

```
from pyspark.sql import functions as F
from pyspark.ml import Pipeline
from pyspark.ml.feature import (
    Imputer,
    StringIndexer,
    OneHotEncoder,
    VectorAssembler
)
df = df.dropDuplicates()
numeric_cols = [
```

```
    "age",
    "ssc_percentage",
    "hsc_percentage",
    "degree_percentage",
    "mba_percentage",
    "internships_count",
    "projects_count",
    "certifications_count",
    "technical_skills_score",
    "soft_skills_score",
    "aptitude_score",
    "communication_score",
    "work_experience_months",
    "leadership_roles",
    "extracurricular_activities",
    "backlogs"
]
categorical_cols = [
    "gender",
    "city_tier",
    "ssc_board",
    "hsc_board",
    "hsc_stream",
    "degree_field",
    "specialization"
]
for col in numeric_cols + ["salary_lpa"]:
    df = df.withColumn(
        col,
        F.when(
            F.isnan(F.col(col).cast("double")) |
            (F.col(col).cast("double") == float("inf")) |
            (F.col(col).cast("double") == float("-inf")),
            F.lit(None).cast("double")
        ).otherwise(
            F.col(col).cast("double")
        )
    )
df = df.withColumn(
    "placed",
    F.col("placed").cast("double")
)
# Keep only valid placement labels
```

```
df = df.filter(
    F.col("placed").isin([0.0, 1.0])
)
# Fill missing categorical values
df = df.fillna({
    col: "Unknown" for col in categorical_cols
})
print("Cleaned dataset records:", df.count())
df.show(5, truncate=False)
```

Listing 4 (notebook cell 4): Train-test split

```
# 80% training, 20% testing
train_df, test_df = df.randomSplit(
    [0.8, 0.2],
    seed=42
)
print("Training records:", train_df.count())
print("Testing records:", test_df.count())
```

Listing 5 (notebook cell 5): Preprocessing pipeline stages

```
from pyspark.ml.feature import Imputer
# Output columns for imputed numerical values
imputed_cols = [
    col + "_imputed" for col in numeric_cols
]
def make_preprocessing_stages():
    # Learn median values from training data
    imputer = Imputer(
        inputCols=numeric_cols,
        outputCols=imputed_cols,
        strategy="median"
    )
    # Convert categorical values to numerical indexes
    indexers = [
        StringIndexer(
            inputCol=col,
            outputCol=col + "_index",
            handleInvalid="keep"
        )
        for col in categorical_cols
    ]
    # One-hot encode categorical variables
    encoder = OneHotEncoder(
```

```
        inputCols=[
            col + "_index" for col in categorical_cols
        ],
        outputCols=[
            col + "_encoded" for col in categorical_cols
        ]
    )
    # Combine numerical and categorical features
    assembler = VectorAssembler(
        inputCols=imputed_cols + [
            col + "_encoded" for col in categorical_cols
        ],
        outputCol="features"
    )
    return [imputer] + indexers + [encoder, assembler]
```

Listing 6 (notebook cell 6): EDA: placement outcome distribution

```
placement_counts = (
    df.groupBy("placed")
      .count()
      .orderBy("placed")
      .toPandas()
)
placement_labels = placement_counts["placed"].map({
    0.0: "Not Placed",
    1.0: "Placed"
})
plt.figure(figsize=(7, 7))
plt.pie(
    placement_counts["count"],
    labels=placement_labels,
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"width": 0.45}
)
plt.title("Placement Outcome Distribution")
plt.tight_layout()
plt.show()
```

Listing 7 (notebook cell 7): EDA: academic scores by placement status

```
academic_cols = [
    "ssc_percentage",
    "hsc_percentage",
```

```
    "degree_percentage"
]
academic_data = (
    df.select("placed", *academic_cols)
      .sample(False, 0.25, seed=42)
      .toPandas()
)
academic_data["Placement"] = academic_data["placed"].map({
    0.0: "Not Placed",
    1.0: "Placed"
})
academic_long = academic_data.melt(
    id_vars="Placement",
    value_vars=academic_cols,
    var_name="Academic Level",
    value_name="Score"
)
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=academic_long,
    x="Academic Level",
    y="Score",
    hue="Placement"
)
plt.title("Academic Score Distribution by Placement Status")
plt.xlabel("Academic Level")
plt.ylabel("Percentage Score")
plt.tight_layout()
plt.show()
```

Listing 8 (notebook cell 8): EDA: correlation heatmap

```
heatmap_cols = [
    "ssc_percentage",
    "hsc_percentage",
    "degree_percentage",
    "internships_count",
    "projects_count",
    "certifications_count",
    "technical_skills_score",
    "soft_skills_score",
    "aptitude_score",
    "communication_score",
    "work_experience_months",
```

```
    "backlogs",
    "placed"
]
heatmap_data = (
    df.select(heatmap_cols)
      .sample(False, 0.25, seed=42)
      .toPandas()
)
correlation = heatmap_data.corr(numeric_only=True)
plt.figure(figsize=(12, 9))
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    linewidths=0.4
)
plt.title("Correlation Heatmap of Student Features")
plt.tight_layout()
plt.show()
```

Listing 9 (notebook cell 9): Classification: training and evaluation of four models

```
from pyspark.ml.classification import (
    LogisticRegression,
    DecisionTreeClassifier,
    LinearSVC,
    NaiveBayes
)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)
classification_models = {
    "Logistic Regression": LogisticRegression(
        labelCol="placed",
        featuresCol="features",
        maxIter=50
    ),
    "Decision Tree": DecisionTreeClassifier(
```

```
        labelCol="placed",
        featuresCol="features",
        maxDepth=5,
        seed=42
    ),
    "SVM": LinearSVC(
        labelCol="placed",
        featuresCol="features",
        maxIter=50
    ),
    "Naive Bayes": NaiveBayes(
        labelCol="placed",
        featuresCol="features",
        modelType="multinomial"
    )
}
classification_results = []
classification_predictions = {}
classification_fitted_models = {}
for model_name, model in classification_models.items():
    pipeline = Pipeline(
        stages=make_preprocessing_stages() + [model]
    )
    fitted_model = pipeline.fit(train_df)
    predictions = fitted_model.transform(test_df)
    # Convert only the required columns to Pandas
    prediction_data = (
        predictions
        .select("placed", "prediction")
        .toPandas()
    )
    y_true = prediction_data["placed"]
    y_pred = prediction_data["prediction"]
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(
        y_true,
        y_pred,
        zero_division=0
    )
    recall = recall_score(
        y_true,
        y_pred,
        zero_division=0
```

```
    )
    f1 = f1_score(
        y_true,
        y_pred,
        zero_division=0
    )
    classification_results.append({
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1 Score": f1
    })
    classification_predictions[model_name] = predictions
    classification_fitted_models[model_name] = fitted_model
    print("\nModel:", model_name)
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1 Score:", round(f1, 4))
classification_results_df = pd.DataFrame(
    classification_results
)
classification_results_df
```

Listing 10 (notebook cell 10): Classification: performance comparison plot

```
metrics_long = classification_results_df.melt(
    id_vars="Model",
    value_vars=[
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],
    var_name="Metric",
    value_name="Score"
)
plt.figure(figsize=(11, 6))
sns.stripplot(
    data=metrics_long,
    x="Score",
    y="Model",
    hue="Metric",
```

```
    dodge=True,
    size=7
)
plt.title("Classification Model Performance Comparison")
plt.xlabel("Evaluation Score")
plt.ylabel("Machine Learning Model")
plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```

Listing 11 (notebook cell 11): Classification: best model and confusion matrix

```
best_model_name = (
    classification_results_df
    .sort_values("F1 Score", ascending=False)
    .iloc[0]["Model"]
)
best_predictions = classification_predictions[
    best_model_name
]
confusion_data = (
    best_predictions
    .select("placed", "prediction")
    .toPandas()
)
cm = confusion_matrix(
    confusion_data["placed"],
    confusion_data["prediction"],
    labels=[0.0, 1.0]
)
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Not Placed", "Placed"],
    yticklabels=["Not Placed", "Placed"]
)
plt.title(f"Confusion Matrix - {best_model_name}")
plt.xlabel("Predicted Placement")
plt.ylabel("Actual Placement")

plt.tight_layout()
```

```
plt.show()
print("Best classification model:", best_model_name)
```

Listing 12 (notebook cell 12): Salary regression: training and evaluation

```
from pyspark.ml.regression import (
    LinearRegression,
    DecisionTreeRegressor
)
from pyspark.ml.evaluation import RegressionEvaluator
# Select placed students with valid salaries
salary_df = df.filter(
    (F.col("placed") == 1.0) &
    F.col("salary_lpa").isNotNull() &
    (F.col("salary_lpa") > 0)
)
print("Students used for salary prediction:", salary_df.count())
# Split salary data
salary_train, salary_test = salary_df.randomSplit(
    [0.8, 0.2],
    seed=42
)
regression_models = {
    "Linear Regression": LinearRegression(
        labelCol="salary_lpa",
        featuresCol="features",
        maxIter=50
    ),
    "Decision Tree Regression": DecisionTreeRegressor(
        labelCol="salary_lpa",
        featuresCol="features",
        maxDepth=5,
        seed=42
    )
}
regression_results = []
regression_predictions = {}
regression_fitted_models = {}
for model_name, model in regression_models.items():
    pipeline = Pipeline(
        stages=make_preprocessing_stages() + [model]
    )
    fitted_model = pipeline.fit(salary_train)
    predictions = fitted_model.transform(salary_test)
```

```
    # Evaluate regression performance
    rmse_evaluator = RegressionEvaluator(
        labelCol="salary_lpa",
        predictionCol="prediction",
        metricName="rmse"
    )
    mae_evaluator = RegressionEvaluator(
        labelCol="salary_lpa",
        predictionCol="prediction",
        metricName="mae"
    )
    r2_evaluator = RegressionEvaluator(
        labelCol="salary_lpa",
        predictionCol="prediction",
        metricName="r2"
    )
    rmse = rmse_evaluator.evaluate(predictions)
    mae = mae_evaluator.evaluate(predictions)
    r2 = r2_evaluator.evaluate(predictions)
    regression_results.append({
        "Model": model_name,
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2
    })
    regression_predictions[model_name] = predictions
    regression_fitted_models[model_name] = fitted_model
    print("\nModel:", model_name)
    print("RMSE:", round(rmse, 3))
    print("MAE:", round(mae, 3))
    print("R2:", round(r2, 3))
regression_results_df = pd.DataFrame(
    regression_results
)
regression_results_df
```

Listing 13 (notebook cell 13): Salary regression: actual vs predicted plot

```
best_regression_name = (
    regression_results_df
    .sort_values("RMSE", ascending=True)
    .iloc[0]["Model"]
)
best_salary_predictions = regression_predictions[
```

```
    best_regression_name
]
salary_plot_data = (
    best_salary_predictions
    .select("salary_lpa", "prediction")
    .toPandas()
)
# Sample for readability
if len(salary_plot_data) > 2000:
    salary_plot_data = salary_plot_data.sample(
        2000,
        random_state=42
    )
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=salary_plot_data,
    x="salary_lpa",
    y="prediction",
    alpha=0.5
)
# Perfect prediction reference line
minimum = min(
    salary_plot_data["salary_lpa"].min(),
    salary_plot_data["prediction"].min()
)
maximum = max(
    salary_plot_data["salary_lpa"].max(),
    salary_plot_data["prediction"].max()
)
plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle="--"
)
plt.title(f"Actual vs Predicted Salary - {best_regression_name}")
plt.xlabel("Actual Salary (LPA)")
plt.ylabel("Predicted Salary (LPA)")
plt.tight_layout()
plt.show()
print("Best salary model:", best_regression_name)
```

Listing 14 (notebook cell 14): K-means: scaling and silhouette comparison

```
from pyspark.ml.clustering import KMeans
```

```
from pyspark.ml.feature import StandardScaler, VectorAssembler
from pyspark.ml.evaluation import ClusteringEvaluator
cluster_features = [
    "ssc_percentage",
    "hsc_percentage",
    "degree_percentage",
    "technical_skills_score",
    "soft_skills_score",
    "aptitude_score",
    "communication_score",
    "internships_count",
    "projects_count",
    "certifications_count",
    "work_experience_months",
    "backlogs"
]
cluster_assembler = VectorAssembler(
    inputCols=cluster_features,
    outputCol="cluster_features"
)
cluster_scaler = StandardScaler(
    inputCol="cluster_features",
    outputCol="scaled_features",
    withMean=True,
    withStd=True
)
cluster_preprocessing = Pipeline(
    stages=[
        cluster_assembler,
        cluster_scaler
    ]
)
# Fit scaling on available clustering data
cluster_features_df = df.select(cluster_features).dropna()
scaled_cluster_data = cluster_preprocessing.fit(
    cluster_features_df
).transform(cluster_features_df)
cluster_evaluator = ClusteringEvaluator(
    featuresCol="scaled_features",
    predictionCol="cluster",
    metricName="silhouette"
)
silhouette_results = []
```

```
for k in [2, 3, 4, 5]:
    model = KMeans(
        featuresCol="scaled_features",
        predictionCol="cluster",
        k=k,
        seed=42
    ).fit(scaled_cluster_data)
    cluster_predictions = model.transform(
        scaled_cluster_data
    )
    score = cluster_evaluator.evaluate(
        cluster_predictions
    )
    silhouette_results.append({
        "K": k,
        "Silhouette Score": score
    })
silhouette_df = pd.DataFrame(silhouette_results)
print(silhouette_df)
best_k = int(
    silhouette_df.sort_values(
        "Silhouette Score",
        ascending=False
    ).iloc[0]["K"]
)
print("Best K:", best_k)
```

Listing 15 (notebook cell 15): Final results summary

```
print("=" * 60)
print("CAMPUS PLACEMENT PREDICTION - FINAL RESULTS")
print("=" * 60)
# 1. Classification results
print("\nCLASSIFICATION RESULTS")
print(
    classification_results_df.sort_values(
        "F1 Score",
        ascending=False
    ).to_string(index=False)
)
print("\nBest Classification Model:", best_model_name)
# 2. Salary regression results
print("\nSALARY REGRESSION RESULTS")
print(
```

```
    regression_results_df.sort_values(
        "RMSE",
        ascending=True
    ).to_string(index=False)
)
print("\nBest Salary Prediction Model:", best_regression_name)
# 3. Student clustering results
print("\nSTUDENT CLUSTERING RESULTS")
# Check whether cluster_summary has been created
if "cluster_summary" in globals():
    cluster_summary.show(truncate=False)
else:
    print("Cluster summary is not available.")
    print("Please run the K-means clustering summary cell first.")
# 4. K-means silhouette comparison
print("\nK-MEANS SILHOUETTE RESULTS")
print(
    silhouette_df.sort_values(
        "Silhouette Score",
        ascending=False
    ).to_string(index=False)
)
# 5. Project completion message
print("\n" + "=" * 60)
print("PROJECT EXECUTION COMPLETED")
print("=" * 60)
```

8. Execution Screenshot

```
Total records: 100000
root
 |-- student_id: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- age: integer (nullable = true)
 |-- city_tier: string (nullable = true)
 |-- ssc_percentage: double (nullable = true)
 |-- ssc_board: string (nullable = true)
 |-- hsc_percentage: double (nullable = true)
 |-- hsc_board: string (nullable = true)
 |-- hsc_stream: string (nullable = true)
 |-- degree_percentage: double (nullable = true)
 |-- degree_field: string (nullable = true)
 |-- mba_percentage: double (nullable = true)
 |-- specialization: string (nullable = true)
 |-- internships_count: integer (nullable = true)
 |-- projects_count: integer (nullable = true)
 |-- certifications_count: integer (nullable = true)
 |-- technical_skills_score: double (nullable = true)
 |-- soft_skills_score: double (nullable = true)
 |-- aptitude_score: double (nullable = true)
 |-- communication_score: double (nullable = true)
 |-- work_experience_months: integer (nullable = true)
 |-- leadership_roles: integer (nullable = true)
 |-- extracurricular_activities: integer (nullable = true)
 |-- backlogs: integer (nullable = true)
 |-- placed: integer (nullable = true)
 |-- salary_lpa: double (nullable = true)
```

```
Cleaned dataset records: 100000
+----------+------+----+---------+-------------+---------+-------------+---------+----------+----------------+-----------+-------------+----------
----+----------------+------------+------------------+--------------------+---------------+------------+----------------+---------------
------+---------------+-------------------------+--------+------+----------+
|student_id|gender|age |city_tier|ssc_percentage|ssc_board|hsc_percentage|hsc_board|hsc_stream|degree_percentage|degree_field|mba_percentage|specializa
tion|internships_count|projects_count|certifications_count|technical_skills_score|soft_skills_score|aptitude_score|communication_score|work_experience_
months|leadership_roles|extracurricular_activities|backlogs|placed|salary_lpa|
+----------+------+----+---------+-------------+---------+-------------+---------+----------+----------------+-----------+-------------+----------
----+----------------+------------+------------------+--------------------+---------------+------------+----------------+---------------
------+---------------+-------------------------+--------+------+----------+
|STU000004 |Female|21.0|Tier 2   |80.4         |CBSE     |77.15        |State    |Science   |56.55           |Engineering |0.0          |None
|2.0             |3.0          |3.0               |6.8                 |7.6            |35.0         |8.3             |31.0
|0.0             |5.0                    |0.0     |1.0   |15.49     |
|STU000007 |Male  |23.0|Tier 2   |51.92        |State    |55.72        |ICSE     |Science   |45.0            |Engineering |0.0          |None
|2.0             |2.0          |0.0               |4.3                 |5.0            |34.4         |5.4             |27.0
|0.0             |0.0                    |6.0     |0.0   |0.0       |
|STU000013 |Female|21.0|Tier 1   |72.54        |State    |79.26        |CBSE     |Commerce  |65.91           |Business    |0.0          |None
|0.0             |1.0          |1.0               |5.5                 |7.1            |83.7         |7.0             |0.0
|0.0             |4.0                    |2.0     |1.0   |10.93     |
|STU000017 |Male  |22.0|Tier 3   |78.63        |State    |93.74        |ICSE     |Science   |95.16           |Other       |0.0          |None
|2.0             |3.0          |2.0               |3.5                 |3.4            |60.7         |4.0             |0.0
|0.0             |6.0                    |0.0     |1.0   |10.41     |
|STU000026 |Female|21.0|Tier 1   |65.23        |CBSE     |69.24        |State    |Science   |63.76           |Engineering |0.0          |None
|2.0             |2.0          |0.0               |4.7                 |6.9            |74.7         |6.9             |0.0
|1.0             |0.0                    |1.0     |1.0   |10.47     |
+----------+------+----+---------+-------------+---------+-------------+---------+----------+----------------+-----------+-------------+----------
----+----------------+------------+------------------+--------------------+---------------+------------+----------------+---------------
------+---------------+-------------------------+--------+------+----------+
only showing top 5 rows
```

```
Students used for salary prediction: 84432

Model: Linear Regression
RMSE: 1.038
MAE: 0.875
R2: 0.834

Model: Decision Tree Regression
RMSE: 1.727
MAE: 1.381
R2: 0.541
```

	Model	RMSE	MAE	R2
0	Linear Regression	1.038099	0.874826	0.834121
1	Decision Tree Regression	1.727064	1.381456	0.540876

```
   K  Silhouette Score
0  2          0.243083
1  3          0.222738
2  4          0.187343
3  5          0.183711
Best K: 2
```

```
Training records: 80200
Testing records: 19800
```

```
Model: Logistic Regression
Accuracy: 0.8607
Precision: 0.878
Recall: 0.9701
F1 Score: 0.9218

Model: Decision Tree
Accuracy: 0.8517
Precision: 0.8577
Recall: 0.9887
F1 Score: 0.9185

Model: SVM
Accuracy: 0.846
Precision: 0.846
Recall: 1.0
F1 Score: 0.9166

Model: Naive Bayes
Accuracy: 0.6971
Precision: 0.8907
Recall: 0.7317
F1 Score: 0.8034
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.860707	0.878040	0.970090	0.921772
1	Decision Tree	0.851667	0.857720	0.988657	0.918546
2	SVM	0.845960	0.845960	1.000000	0.916553
3	Naive Bayes	0.697071	0.890698	0.731701	0.803409

```
============================================================
CAMPUS PLACEMENT PREDICTION - FINAL RESULTS
============================================================

CLASSIFICATION RESULTS
              Model  Accuracy  Precision   Recall  F1 Score
Logistic Regression  0.860707   0.878040 0.970090  0.921772
      Decision Tree  0.851667   0.857720 0.988657  0.918546
                SVM  0.845960   0.845960 1.000000  0.916553
        Naive Bayes  0.697071   0.890698 0.731701  0.803409

Best Classification Model: Logistic Regression

SALARY REGRESSION RESULTS
                   Model     RMSE      MAE       R2
       Linear Regression 1.038099 0.874826 0.834121
Decision Tree Regression 1.727064 1.381456 0.540876

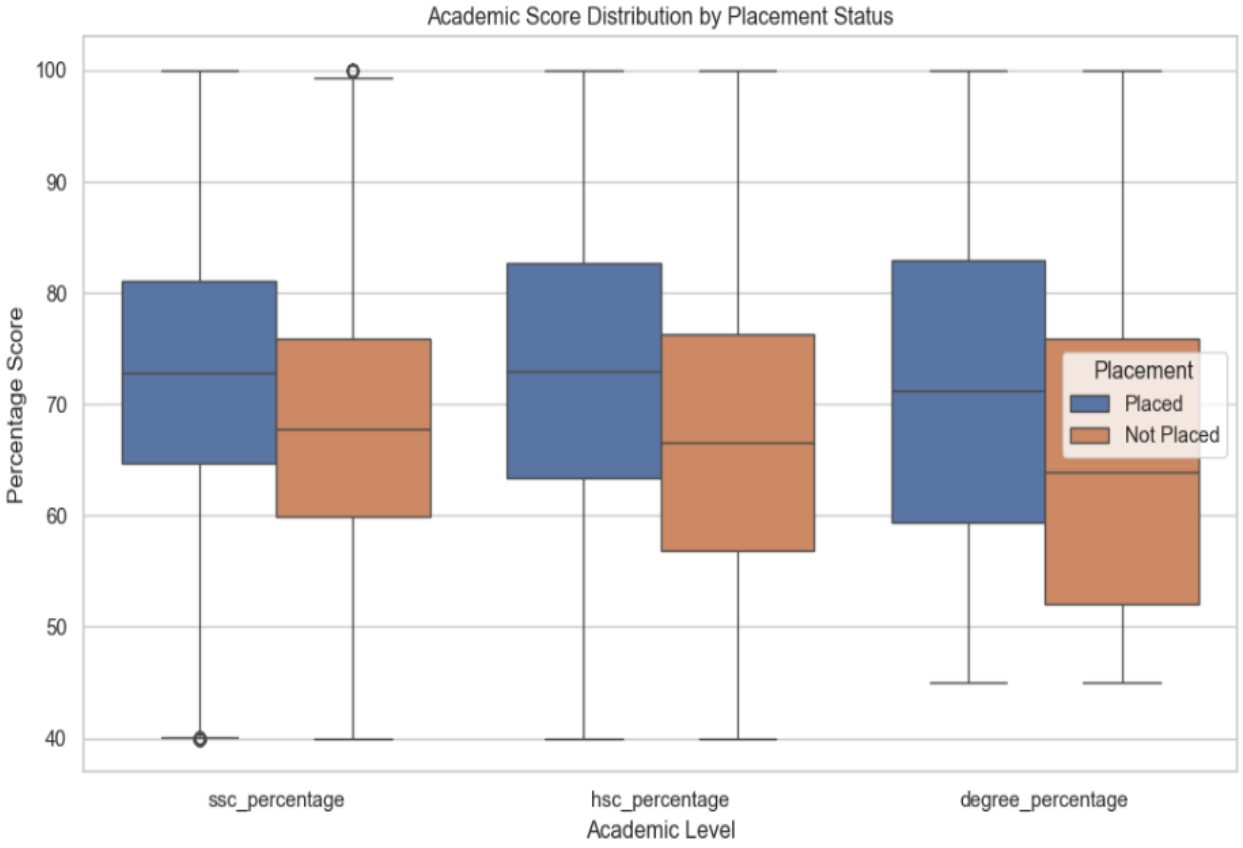
Best Salary Prediction Model: Linear Regression

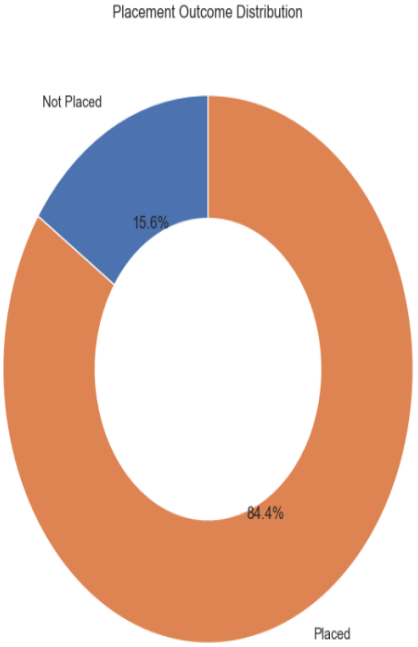
STUDENT CLUSTERING RESULTS
Cluster summary is not available.
Please run the K-means clustering summary cell first.

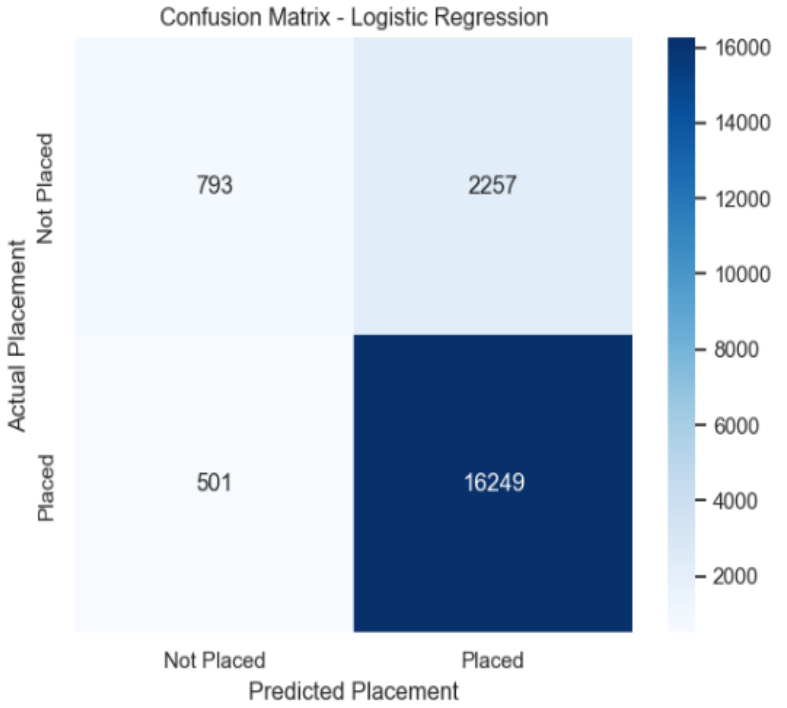
K-MEANS SILHOUETTE RESULTS
 K  Silhouette Score
 2          0.243083
 3          0.222738
 4          0.187343
 5          0.183711

============================================================
PROJECT EXECUTION COMPLETED
============================================================
```

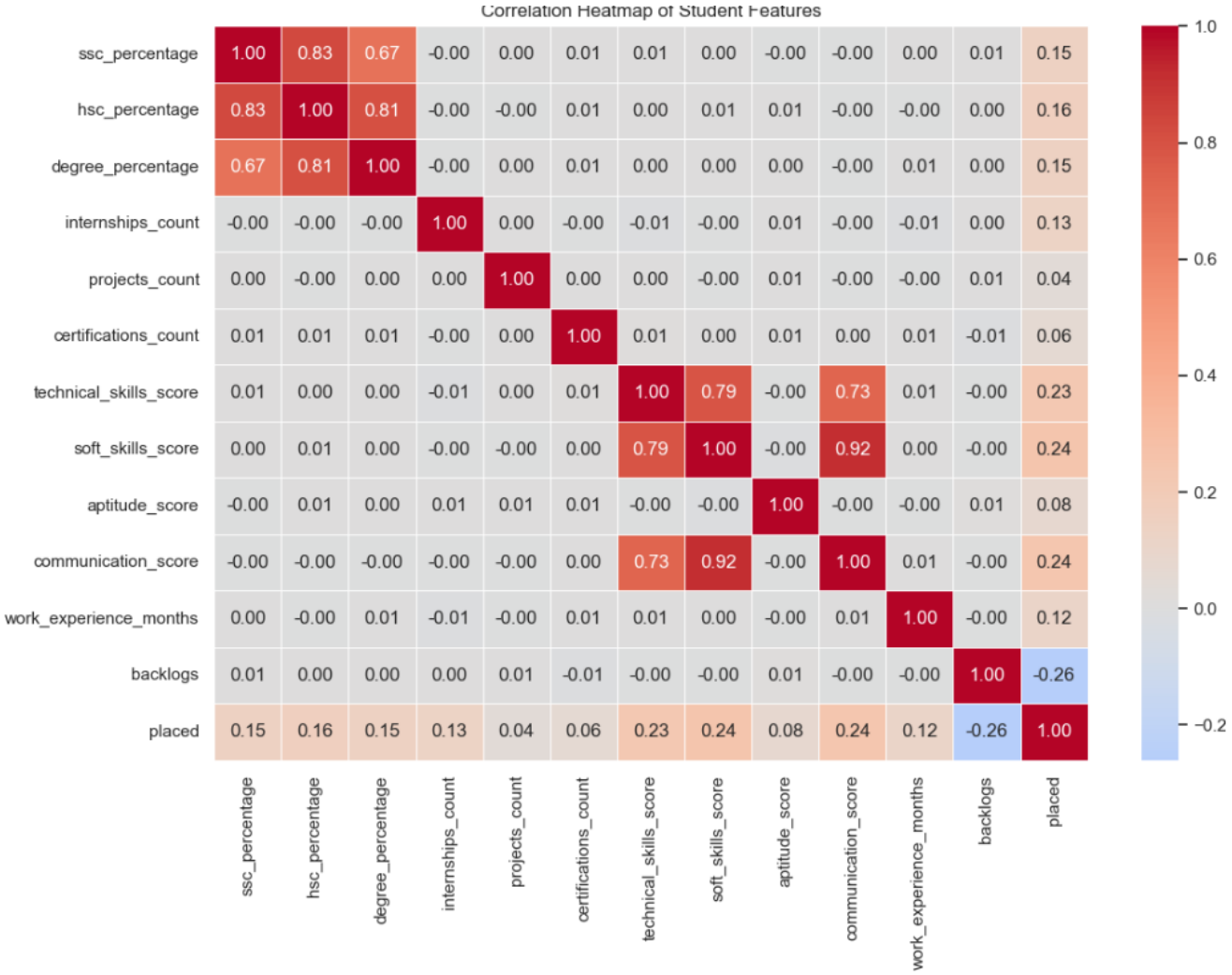
9. Output / Result Screenshots

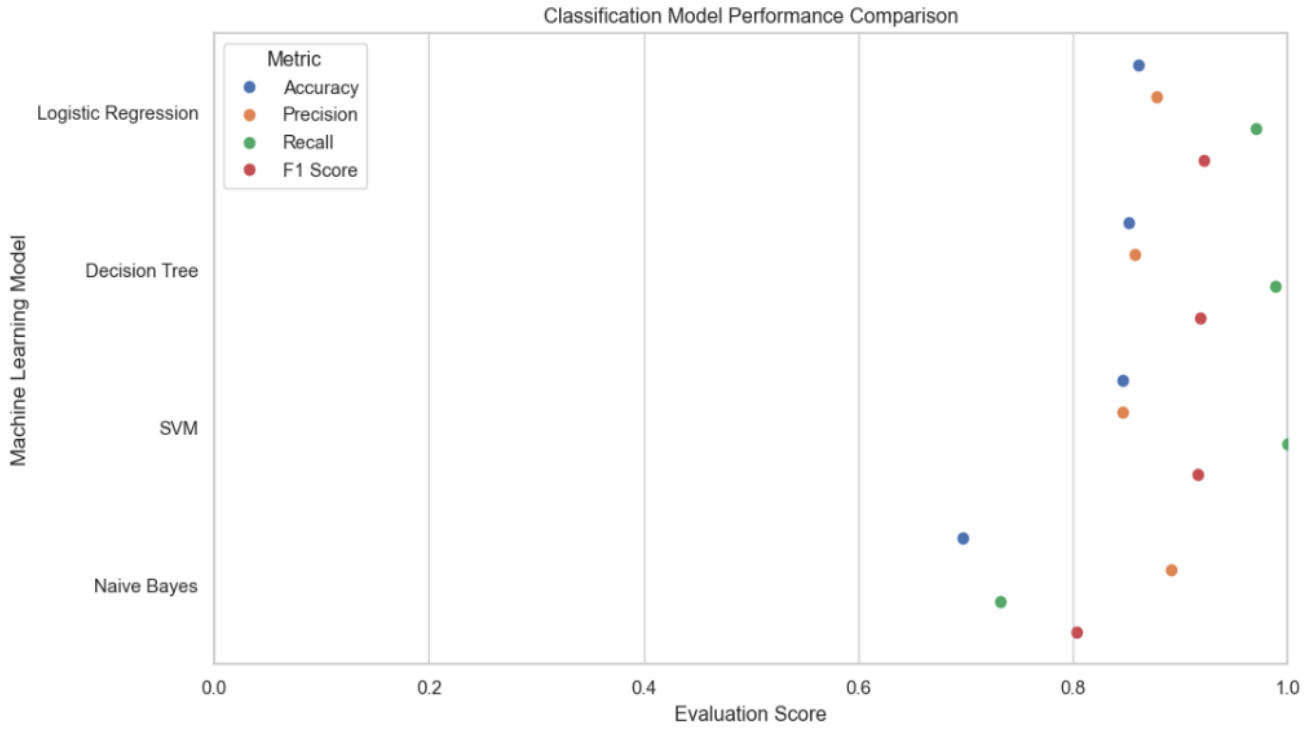


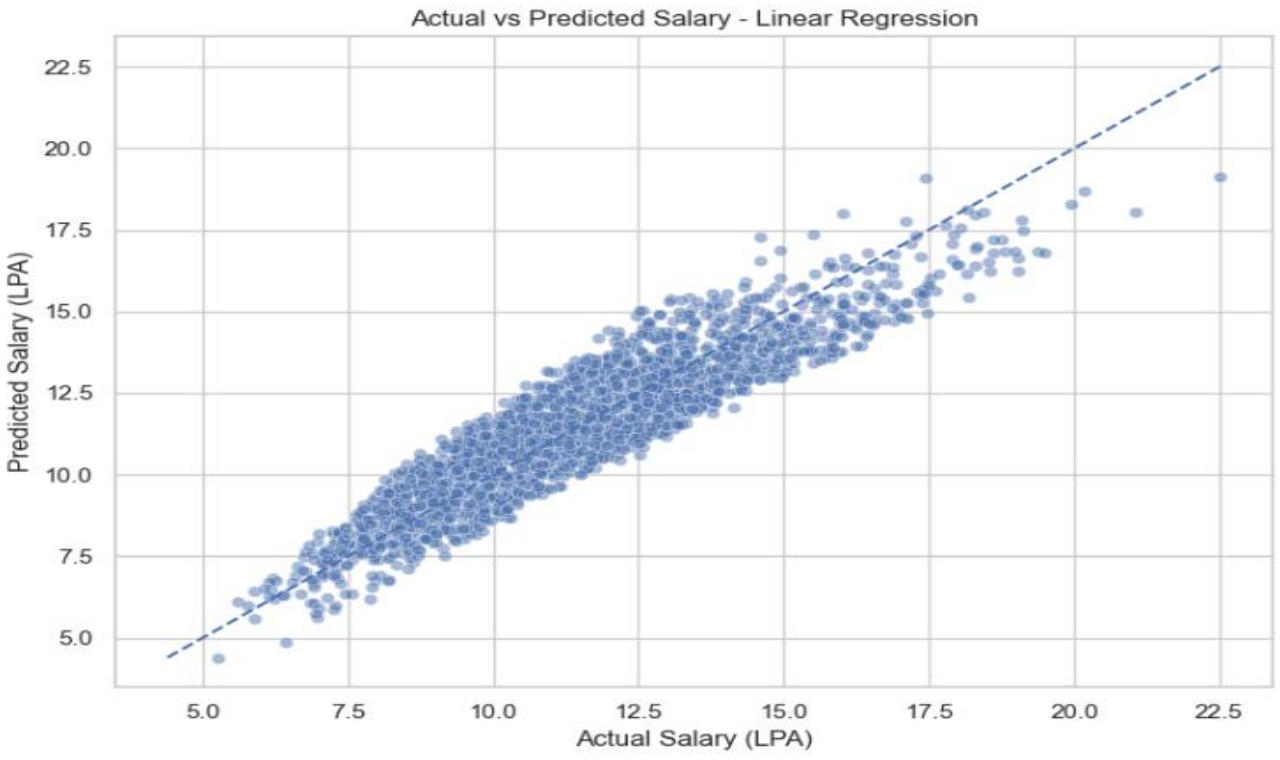





```
Best classification model: Logistic Regression
```








```
Best salary model: Linear Regression
```

10. Result Analysis

10.1 Classification Analysis

The notebook chooses the best model by the highest F1-score, and this criterion is used here. Logistic Regression ranks first with an F1-score of 0.9218, followed by the Decision Tree (0.9185), SVM (0.9166) and Naive Bayes (0.8034) (Table 5). Logistic Regression also has the highest accuracy (0.8607). The gap between the first three models is small (about 0.5 percentage points in F1-score) and each model was trained once on a single split, so the notebook gives no evidence that this ordering would hold on another split. Naive Bayes is clearly the weakest in accuracy (0.6971) and recall (0.7317), although it has the highest precision (0.8907).

The confusion matrix of Logistic Regression (Figure 8) on the 19,800 test students contains 793 true negatives, 2,257 false positives, 501 false negatives and 16,249 true positives. The model therefore finds most

placed students (recall 0.9701), but of the 3,050 students who were not placed it correctly identifies only 793, about 26%, which I calculated from the matrix. This matters because identifying students at risk of not being placed is the purpose of the classifier.

The class imbalance has to be kept in mind. The test set contains 16,750 placed students (84.6%). SVM has recall 1.0 and precision equal to its accuracy (0.8460), which is what would happen if it predicted every student as placed, and this equals the share of placed students in the test set. The accuracy of Logistic Regression (0.8607) is only about 1.5 percentage points above this level, so the F1-score for the placed class, which is the notebook's criterion, does not by itself show how well not-placed students are detected. Class weights, resampling and hyperparameter tuning were not used in the notebook.

10.2 Salary Regression Analysis

Linear Regression performs better than Decision Tree Regression on all three measures (Table 6): its RMSE is 1.038 LPA against 1.727 LPA, its MAE is 0.875 LPA against 1.381 LPA, and its RZ is 0.834 against 0.541. It is selected because it has the lowest RMSE. An RZ of 0.834 means that about 83% of the salary variance in the test set is explained by the model. In Figure 12 the points follow the dashed line closely for mid-range salaries, while for the highest actual salaries (above about 17 LPA) most points lie below the line, so large salaries tend to be under-predicted. The depth-5 tree is a much simpler model than the linear one for this data and this may be one reason for its lower scores, but the notebook does not test this.

10.3 Clustering Analysis

The silhouette score is highest for K = 2 (0.2431) and falls as K increases: 0.2227 for K = 3, 0.1873 for K = 4 and 0.1837 for K = 5 (Table 7), so K = 2 is the number of clusters selected by the notebook. Silhouette values lie between −1 and 1, and values near 0 mean that clusters overlap. All four scores are below 0.25, which suggests that the students do not form clearly separated groups on these 12 features. The notebook does not fit a final model with K = 2 or print cluster sizes, average feature values or placement rates per cluster, so no description of the clusters is made in this report.

10.4 Overall Analysis

- **Objective 1:** achieved in local mode. The 100,000 records were processed with Spark DataFrames and ML pipelines, but there was no multi-node cluster and no scalability measurement.
- **Objective 2:** achieved. Four classifiers were trained and compared, and Logistic Regression was best by F1-score, with the limitations in Section 11.1.
- **Objective 3:** achieved. Linear Regression was better than Decision Tree Regression on RZ, RMSE and MAE.
- **Objective 4:** partly achieved. All required metrics were calculated and used to select models, and the confusion matrix was drawn for the best classifier. The notebook uses fixed settings and does not tune hyperparameters, so no optimization beyond model comparison was done.
- **Objective 5:** partly achieved. The exploratory analysis, correlation heatmap and K-means silhouette comparison were completed, but the student cluster profiles were not produced, so employability profiles are not described.

11. Conclusion

This project used PySpark and Spark MLlib to analyze 100,000 student records. The data was loaded into Spark DataFrames, cleaned, encoded and passed through Spark ML pipelines in local mode. For placement classification, Logistic Regression, Decision Tree, SVM and Naive Bayes were compared with accuracy, precision, recall, F1-score and a confusion matrix, and Logistic Regression gave the highest F1-score (0.9218). For salary prediction on 84,432 placed students, Linear Regression (RZ = 0.834, RMSE = 1.038, MAE = 0.875) performed better than Decision Tree Regression. K-means was run for K = 2 to 5 and the silhouette score selected K = 2 (0.2431), but the resulting clusters were not summarized.

According to the implementation, Objectives 1, 2 and 3 were achieved, Objective 4 was achieved for model evaluation and comparison without hyperparameter tuning, and Objective 5 was achieved for exploratory analysis and cluster-number selection but not for describing student profiles. The results come from a synthetic dataset and a single train-test split, so they should be read as results for this dataset only.